《数据结构基础》(FDS) 专项练习题

Part 2: Algorithm Analysis & Lists/Stacks/Queues

Based on ZJU Past Midterm Exams

说明：本卷包含【第一章：算法分析 (Algorithm Analysis)】与【第二章：线性表、栈与队列 (Lists, Stacks, Queues)】的独立专项训练。根据要求，本次专项练习 ** 题量已加大 **，覆盖历年核心高频考点。

Chapter 1: Algorithm Analysis 专项练习

一、判断题 (True or False)

1-1. $N^2 \log N$ and $N \log N^2$ have the same speed of growth. (T / F)

1-2. 2^N and N^N have the same speed of growth. (T / F)

1-3. $N(\log N)^2$ is $O(N^2)$. (T / F)

1-4. For any algorithm, the best-case time complexity is always lower bound $\Omega(1)$. (T / F)

1-5. If $T_1(n) = O(f(n))$ and $T_2(n) = O(g(n))$, then $T_1(n) + T_2(n) = O(\max(f(n), g(n)))$. (T / F)

1-6. The time complexity of calculating the N-th Fibonacci number using simple recursion 'Fib(N) = Fib(N-1) + Fib(N-2)' is $O(N)$. (T / F)

1-7. A program is written in some programming language, and it must satisfy the "Finiteness" property just like an algorithm. (T / F)

二、选择题 (Multiple Choice)

2-1. The time complexity of the following code fragment is:

```
int sum = 0;
for(int i = 0; i < N; i++)
    for(int j = i; j > 0; j--)
        sum++;
```

A. $O(N)$

B. $O(N \log N)$

C. $O(N^2)$

D. $O(N^3)$

2-2. Given the recurrence relation $T(N) = 2T(N/2) + cN$ with $T(1) = O(1)$, the time complexity is bounded by:

A. $O(N)$

B. $O(N \log N)$

C. $O(N^2)$

D. $O(\log N)$

2-3. The fastest algorithm for finding the Maximum Subsequence Sum problem has a time complexity of:

A. $O(N^3)$

B. $O(N^2)$

C. $O(N \log N)$

D. $O(N)$

2-4. If the most commonly used operations are to visit a random position and to insert and delete the last element in a linear list, then which storage method works the fastest?

A. Sequential storage (Array)

B. Singly linked list

C. Doubly linked list

D. Circular linked list

三、程序填空题 (Fill in the blanks)

3-1. The following function implements the Binary Search algorithm to find an element `X` in a sorted array `A` of size `N`. Please complete the code.

```
int BinarySearch( const ElementType A[], ElementType X, int N ) {
    int Low, Mid, High;
    Low = 0; High = ________(1)________;
    while ( ________(2)________ ) {
        Mid = ( Low + High ) / 2;
        if ( A[ Mid ] < X )
            ________(3)________;
        else if ( A[ Mid ] > X )
```

```
            ________(4)________;
        else
            return Mid;   /* Found */
    }
    return -1; /* NotFound */
}
```

四、函数题 (Programming)

4-1. Maximum Subsequence Sum ($O(N)$ Algorithm)

Write the $O(N)$ function to find the maximum subsequence sum of a given integer array. If all integers are negative, the maximum sum is defined as 0.

```
int MaxSubsequenceSum( const int A[], int N );
```

Chapter 2: Lists, Stacks, Queues 专项练习

一、判断题 (True or False)

1-1. If keys are pushed onto a stack in the order {1, 2, 3, 4, 5}, then it is impossible to obtain the output sequence {3, 4, 1, 2, 5}. (T / F)

1-2. For a sequentially stored linear list of length N, the time complexities for deleting the first element and inserting the last element are $O(1)$ and $O(N)$, respectively. (T / F)

1-3. If a linear list is represented by a linked list, the addresses of the elements in the memory must be consecutive. (T / F)

1-4. In a circular queue implemented by an array with a maximum capacity of `MaxSize`, if `Front` and `Rear` are the pointers, the number of elements is always `(Rear - Front) % MaxSize`. (T / F)

1-5. Given the input sequence onto a stack as $\{1, 2, 3, \ldots, N\}$. If the first output is i, then the j-th output must be $i - j + 1$. (T / F)

1-6. The Cursor implementation of linked lists does not require calls to `malloc` and `free` during the regular node insertion and deletion operations. (T / F)

1-7. To implement a queue using a singly linked list, it is more efficient to maintain a tail pointer and enqueue at the tail, and dequeue at the head. (T / F)

二、选择题 (Multiple Choice)

2-1. Given that the pushing sequence of a stack is $\{1, 2, \ldots, n\}$ and the popping sequence is $\{x_1, x_2, \ldots, x_n\}$. If $x_2 = n$, we can obtain how many different possible popping sequences?

A. 1

B. 2

C. $n - 1$

D. n

2-2. A circular queue has an array size of M. If the front pointer is `Front` and the rear pointer is `Rear`, what is the number of elements currently in the queue?

A. `(Rear - Front + M) % M`

B. `(Rear - Front) % M`

C. `(Front - Rear + M) % M`

D. `(Rear - Front + 1) % M`

2-3. Which of the following data structures can be used to perform both insertions and deletions at both ends in $O(1)$ time?

A. Singly linked circular list with a tail pointer

B. Doubly linked circular list with a dummy head node

C. Singly linked list

D. Array-based list without shifting

2-4. When evaluating the postfix expression `5 4 6 + * 4 9 3 / + *`, what is the maximum number of elements in the stack during the evaluation?

A. 3

B. 4

C. 5

D. 6

2-5. For a queue represented by a circular array of `MaxSize`, which of the following is typically used to judge if the queue is full? (Assume we leave one empty space).

A. `Front == Rear`

B. `(Rear + 1) % MaxSize == Front`

C. `Rear == MaxSize - 1`

D. `Front == (Rear + 1) % MaxSize`

三、程序填空题 (Fill in the blanks)

3-1. The following code is to enqueue an element `X` into a circular queue `Q` implemented by an array. Assume the capacity is `Q->Capacity` and we leave one empty slot to differentiate full from empty.

```
bool Enqueue( ElementType X, Queue Q ) {
    if ( ________(1)________ ) {
        printf("Queue is Full\n");
        return false;
    } else {
        ________(2)________;
        Q->Array[ Q->Rear ] = X;
        return true;
    }
}
```

四、函数题 (Programming)

4-1. Reverse Linked List

Write a function to return the reversed linked list of `L`, with a dummy header.

```
List Reverse( List L );
```

Note: `List` is defined as a pointer to a struct node with `Element` and `Next`. The input `L` contains a dummy head node, and the returned list should also have a dummy head node.

参考答案与解析

Chapter 1: Algorithm Analysis

一、判断题

1. **F**. $N \log N^2 = 2N \log N = O(N \log N)$. It grows much slower than $N^2 \log N$.
2. **F**. N^N grows much faster than 2^N.
3. **T**. Since $(\log N)^2$ grows slower than N, $N(\log N)^2$ grows slower than $N \cdot N = N^2$.
4. **T**. Any algorithm needs at least a constant amount of time $O(1)$ to do anything.
5. **T**. This is the standard addition rule of Big-O notation.
6. **F**. The simple recursive algorithm for Fibonacci takes exponential time $O(2^N)$ because of massive redundant calculations.
7. **F**. An algorithm must satisfy "Finiteness" (terminate in finite steps), but a program (like an Operating System) may run infinitely.

二、选择题

1. **C**. The outer loop runs N times. The inner loop runs i times. The total operations are $1 + 2 + \cdots + N - 1 = \frac{N(N-1)}{2} = O(N^2)$.
2. **B**. By Master Theorem or recursion tree: $T(N) = 2T(N/2) + O(N)$ yields $O(N \log N)$.
3. **D**. The dynamic programming / greedy approach solves it in a single pass $O(N)$.
4. **A**. Sequential storage (Array) takes $O(1)$ for random access, $O(1)$ for inserting at the very end, and $O(1)$ for deleting the last element.

三、程序填空题

1. `N - 1`
2. `Low <= High`
3. `Low = Mid + 1`
4. `High = Mid - 1`

四、函数题

```
int MaxSubsequenceSum( const int A[], int N ) {
    int ThisSum = 0, MaxSum = 0;
    int i;
    for( i = 0; i < N; i++ ) {
        ThisSum += A[i];
        if( ThisSum > MaxSum )
            MaxSum = ThisSum;
        else if( ThisSum < 0 )
            ThisSum = 0;
    }
    return MaxSum;
}
```

Chapter 2: Lists, Stacks, Queues

一、判断题

1. **T**. Stack LIFO rule. If 4 is popped, 1 and 2 are in the stack (1 at bottom, 2 above). You must pop 2 before 1.

2. **F**. For an Array, deleting the first element requires moving $N-1$ elements forward, so it's $O(N)$. Inserting the last element is $O(1)$.

3. **F**. Linked lists dynamically allocate memory via pointers, so addresses do not need to be consecutive.

4. **F**. The formula is `(Rear - Front + MaxSize) % MaxSize` because `Rear - Front` can be negative.

5. **F**. If the first output is i, the subsequent outputs depend entirely on when items are popped. The j-th output is not strictly fixed to a mathematical formula.

6. **T**. The Cursor implementation allocates a huge array upfront and manually manages "free" and "used" lists via array indices instead of OS calls.

7. **T**. A queue needs FIFO. Enqueuing at the tail is $O(1)$ if you have a tail pointer. Dequeuing at the head is $O(1)$.

二、选择题

1. **A**. If $x_2 = n$, it means the first popped element is something, then n is popped. Since n is pushed last, the only way n is popped 2nd is if $n-1$ elements are pushed, one is popped (x_1), then n is pushed and popped. The remaining items in the stack are fixed and must pop in strictly reverse order. So only 1 sequence exists.

2. **A**. Formula for circular queue size is `(Rear - Front + M) % M`.

3. **B**. A doubly linked circular list with a dummy head node allows $O(1)$ access, insertion, and deletion at both the front and the back.

4. **B**. Evaluating `5 4 6 + * 4 9 3 / + *`: Push 5 (size=1), Push 4 (size=2), Push 6 (size=3). '+': Pop 6, 4. Push 10 (size=2). '*': Pop 10, 5. Push 50 (size=1). Push 4 (size=2), Push 9 (size=3), Push 3 (size=4). <– **Max size is 4** '/': Pop 3, 9. Push 3 (size=3). '+': Pop 3, 4. Push 7 (size=2). '*': Pop 7, 50. Push 350 (size=1).

5. **B/D** (Depending on convention, standard ZJU FDS is: `(Rear + 1) % MaxSize == Front`).

三、程序填空题

1. `( Q->Rear + 1 ) % Q->Capacity == Q->Front` (判断队列是否已满)

2. `Q->Rear = ( Q->Rear + 1 ) % Q->Capacity` (更新 Rear 指针)

四、函数题

```
List Reverse( List L ) {
    List prev = NULL;
    List curr = L->Next;  /* Skip dummy head */
    List nextTemp;

    while ( curr != NULL ) {
        nextTemp = curr->Next; /* Save next node */
        curr->Next = prev;     /* Reverse pointer */
        prev = curr;           /* Move prev forward */
        curr = nextTemp;       /* Move curr forward */
    }

    L->Next = prev; /* Attach the reversed list to the dummy head */
    return L;
}
```